PRACTICAL # 04

Write the program in C/C++/Java to implement Preemptive and Non-Preemptive Shortest Job First CPU Scheduling Algorithm.

Date of allocation:29-30 Jan 2026 **Date of submission: 05-06 Feb 2026**

C Program – SJF (Non-Preemptive)

```
#include <stdio.h>

struct Process {
   int pid, at, bt, ct, tat, wt;
};

int main() {
   int n;
   printf("Enter number of processes: ");
   scanf("%d", &n);

   struct Process p[n], temp;
   int completed = 0, current_time = 0;
   float total_tat = 0, total_wt = 0;

   for(int i = 0; i < n; i++) {
      printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
      p[i].pid = i+1;
      scanf("%d %d", &p[i].at, &p[i].bt);
   }

   while(completed < n) {
      int idx = -1;
      int min_bt = 9999;

      for(int i = 0; i < n; i++) {
         if(p[i].at <= current_time && p[i].ct == 0) {
            if(p[i].bt < min_bt) {
               min_bt = p[i].bt;
               idx = i;
            }
         }
      }

      if(idx != -1) {
         current_time += p[idx].bt;
         p[idx].ct = current_time;
         p[idx].tat = p[idx].ct - p[idx].at;
         p[idx].wt = p[idx].tat - p[idx].bt;

         total_tat += p[idx].tat;
```

```
            total_wt += p[idx].wt;
            completed++;
        } else {
            current_time++;
        }
    }

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt,
            p[i].ct, p[i].tat, p[i].wt);
    }

    printf("\nAverage Turnaround Time = %.2f", total_tat/n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt/n);

    return 0;
}
```

C Program – SJF (Preemptive – SRTF)

```
#include <stdio.h>

int main() {
    int n, completed = 0, current_time = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n];
    float total_tat = 0, total_wt = 0;

    for(int i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];  // remaining time
    }

    while(completed < n) {
        int idx = -1;
        int min_rt = 9999;

        for(int i = 0; i < n; i++) {
            if(at[i] <= current_time && rt[i] > 0) {
                if(rt[i] < min_rt) {
                    min_rt = rt[i];
                    idx = i;
                }
            }
        }

        if(idx != -1) {
            rt[idx]--;
```

```
            current_time++;

            if(rt[idx] == 0) {
                completed++;
                ct[idx] = current_time;
                tat[idx] = ct[idx] - at[idx];
                wt[idx] = tat[idx] - bt[idx];

                total_tat += tat[idx];
                total_wt += wt[idx];
            }
        } else {
            current_time++;
        }
    }

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            i+1, at[i], bt[i],
            ct[i], tat[i], wt[i]);
    }

    printf("\nAverage Turnaround Time = %.2f", total_tat/n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt/n);
    return 0;
}
```

Non-Preemptive SJF (C++)

```
            #include <iostream>
#include <vector>
#include <iomanip>
using namespace std;

struct Process {
    int pid, at, bt, ct = 0, tat = 0, wt = 0;
};

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    vector<Process> p(n);

    for(int i = 0; i < n; i++) {
        cout << "Enter Arrival Time and Burst Time for P" << i+1 << ": ";
        p[i].pid = i+1;
        cin >> p[i].at >> p[i].bt;
    }
```

```
    int completed = 0, current_time = 0;
    float total_tat = 0, total_wt = 0;

    while(completed < n) {
        int idx = -1;
        int min_bt = 1e9;

        for(int i = 0; i < n; i++) {
            if(p[i].at <= current_time && p[i].ct == 0) {
                if(p[i].bt < min_bt) {
                    min_bt = p[i].bt;
                    idx = i;
                }
            }
        }

        if(idx != -1) {
            current_time += p[idx].bt;
            p[idx].ct = current_time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;

            total_tat += p[idx].tat;
            total_wt += p[idx].wt;
            completed++;
        } else {
            current_time++;
        }
    }

    cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
    for(int i = 0; i < n; i++) {
        cout << "P" << p[i].pid << "\t"
             << p[i].at << "\t"
             << p[i].bt << "\t"
             << p[i].ct << "\t"
             << p[i].tat << "\t"
             << p[i].wt << "\n";
    }

    cout << fixed << setprecision(2);
    cout << "\nAverage Turnaround Time: " << total_tat/n;
    cout << "\nAverage Waiting Time: " << total_wt/n << endl;

    return 0;
}
```

Preemptive SJF (SRTF) – C++

```
        #include <iostream>
#include <vector>
#include <iomanip>
using namespace std;

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    vector<int> at(n), bt(n), rt(n), ct(n), tat(n), wt(n);

    for(int i = 0; i < n; i++) {
        cout << "Enter Arrival Time and Burst Time for P" << i+1 << ": ";
        cin >> at[i] >> bt[i];
        rt[i] = bt[i];   // Remaining Time
    }

    int completed = 0, current_time = 0;
    float total_tat = 0, total_wt = 0;

    while(completed < n) {
        int idx = -1;
        int min_rt = 1e9;

        for(int i = 0; i < n; i++) {
            if(at[i] <= current_time && rt[i] > 0) {
                if(rt[i] < min_rt) {
                    min_rt = rt[i];
                    idx = i;
                }
            }
        }

        if(idx != -1) {
            rt[idx]--;
            current_time++;

            if(rt[idx] == 0) {
                completed++;
                ct[idx] = current_time;
                tat[idx] = ct[idx] - at[idx];
                wt[idx] = tat[idx] - bt[idx];

                total_tat += tat[idx];
                total_wt += wt[idx];
            }
        } else {
            current_time++;
```

```
        }
    }

    cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
    for(int i = 0; i < n; i++) {
        cout << "P" << i+1 << "\t"
             << at[i] << "\t"
             << bt[i] << "\t"
             << ct[i] << "\t"
             << tat[i] << "\t"
             << wt[i] << "\n";
    }

    cout << fixed << setprecision(2);
    cout << "\nAverage Turnaround Time: " << total_tat/n;
    cout << "\nAverage Waiting Time: " << total_wt/n << endl;

    return 0;
}
```

Non-Preemptive SJF – Java

```
import java.util.*;

class NonPreemptiveSJF {

    static class Process {
        int pid, at, bt, ct = 0, tat = 0, wt = 0;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of processes: ");
        int n = sc.nextInt();

        Process[] p = new Process[n];

        for (int i = 0; i < n; i++) {
            p[i] = new Process();
            p[i].pid = i + 1;

            System.out.print("Enter Arrival Time and Burst Time for P" + (i + 1) + ": ");
            p[i].at = sc.nextInt();
            p[i].bt = sc.nextInt();
        }

        int completed = 0, currentTime = 0;
        float totalTAT = 0, totalWT = 0;

        while (completed < n) {
```

```
        int idx = -1;
        int minBT = Integer.MAX_VALUE;

        for (int i = 0; i < n; i++) {
            if (p[i].at <= currentTime && p[i].ct == 0) {
                if (p[i].bt < minBT) {
                    minBT = p[i].bt;
                    idx = i;
                }
            }
        }

        if (idx != -1) {
            currentTime += p[idx].bt;
            p[idx].ct = currentTime;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;

            totalTAT += p[idx].tat;
            totalWT += p[idx].wt;
            completed++;
        } else {
            currentTime++;
        }
    }

    System.out.println("\nPID\tAT\tBT\tCT\tTAT\tWT");
    for (int i = 0; i < n; i++) {
        System.out.println("P" + p[i].pid + "\t" +
                p[i].at + "\t" +
                p[i].bt + "\t" +
                p[i].ct + "\t" +
                p[i].tat + "\t" +
                p[i].wt);
    }

    System.out.printf("\nAverage Turnaround Time: %.2f", totalTAT / n);
    System.out.printf("\nAverage Waiting Time: %.2f\n", totalWT / n);

    sc.close();
  }
}
```

Preemptive SJF (SRTF) – Java

```
        import java.util.*;

class PreemptiveSJF {

  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter number of processes: ");
int n = sc.nextInt();

int[] at = new int[n];
int[] bt = new int[n];
int[] rt = new int[n];
int[] ct = new int[n];
int[] tat = new int[n];
int[] wt = new int[n];

for (int i = 0; i < n; i++) {
    System.out.print("Enter Arrival Time and Burst Time for P" + (i + 1) + ": ");
    at[i] = sc.nextInt();
    bt[i] = sc.nextInt();
    rt[i] = bt[i]; // Remaining Time
}

int completed = 0, currentTime = 0;
float totalTAT = 0, totalWT = 0;

while (completed < n) {
    int idx = -1;
    int minRT = Integer.MAX_VALUE;

    for (int i = 0; i < n; i++) {
        if (at[i] <= currentTime && rt[i] > 0) {
            if (rt[i] < minRT) {
                minRT = rt[i];
                idx = i;
            }
        }
    }

    if (idx != -1) {
        rt[idx]--;
        currentTime++;

        if (rt[idx] == 0) {
            completed++;
            ct[idx] = currentTime;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];

            totalTAT += tat[idx];
            totalWT += wt[idx];
        }
    } else {
        currentTime++;
    }
}
```

```
        System.out.println("\nPID\tAT\tBT\tCT\tTAT\tWT");
        for (int i = 0; i < n; i++) {
            System.out.println("P" + (i + 1) + "\t" +
                    at[i] + "\t" +
                    bt[i] + "\t" +
                    ct[i] + "\t" +
                    tat[i] + "\t" +
                    wt[i]);
        }

        System.out.printf("\nAverage Turnaround Time: %.2f", totalTAT / n);
        System.out.printf("\nAverage Waiting Time: %.2f\n", totalWT / n);

        sc.close();
    }
}
```